Practical 7

Sr. No	Practical Question	Type
1	Define a standard function square(n) using the def keyword that takes a single number as input and returns its square. Test it with the number 7 and print the result.	Easy
2	Create a function add_three(a, b, c) that takes three numbers and returns their sum. Call it with values 5, 10, and 15.	Easy
3	Write a function is_even(number) that returns True if a given integer is even, and False otherwise.	Easy
4	Write a function find_max(a, b) that compares two numbers using an if-else statement and returns the larger number.	Easy
5	Create a tuple of coordinate pairs points = ((1, 2), (4, 1), (3, 8), (2, 5)). Write a custom function get_second(item) and pass it as the key parameter to sorted() to sort the tuples by their second element.	Easy
6	Create a dictionary scores = {"Alice": 88, "Bob": 95, "Charlie": 78}. Write a function get_score(student_name) that returns scores[student_name] and use it with sorted() to print the names sorted by their scores in descending order.	Easy

7	Given a list of words `words = ["python", "java", "c", "javascript", "go"]`, write a custom function `get_length(word)` that returns `len(word)` and use it to sort the list by string length.	**Easy**
8	Create a tuple `dimensions = (10, 20, 30)`. Attempt to reassign `dimensions[0] = 15` inside a `try-except` block and catch the `TypeError` with a custom error message.	**Easy**
9	Given a dictionary `item = {"name": "Monitor", "price": 12000}`, use an `if-else` statement with the `in` operator to check if `"discount"` exists. If not, add `"discount": 0`.	**Easy**
10	Use a `for` loop to iterate directly over the keys of `inventory = {"apple": 50, "orange": 30, "grape": 20}` and print `"We have [value] units of [key]"`.	**Easy**
11	Write a function `check_length(text)` that returns `"Short"` if `len(text) < 5`, and `"Long"` otherwise. Test it with various string inputs.	**Moderate**
12	Write a function `extract_odds(numbers_list)` that takes a list of integers, loops through it, and returns a new list containing only the odd numbers. Test it with `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`.	**Moderate**
13	Write a function `celsius_to_fahrenheit(c)` that converts a Celsius temperature to Fahrenheit using $(F = C \times 1.8 + 32)$. Use a `for` loop to convert a list `celsius = [0, 20, 37, 100]` into a new list of Fahrenheit values.	**Moderate**
14	Given `dict1 = {"x": 10, "y": 20}` and `dict2 = {"y": 25, "z": 30}`, use `.update()` to merge `dict2` into `dict1`. Print the resulting dictionary.	**Moderate**

15	Given session = {"user": "guest", "id": 404, "active": False}, use .popitem() in a while loop to clear the dictionary item-by-item, printing each popped key-value pair.	**Moderate**
16	Create a tuple data = (10, 20, 30, 40, 50). Use tuple slicing to extract a sub-tuple containing only (20, 30, 40) and reverse it using negative slicing [::-1].	**Moderate**
17	Take a sentence input from the user. Write a program using a loop and a dictionary to count how many times each word appears in the sentence.	**Moderate**
18	Given employees = {"ID1": "Ana", "ID2": "Ben"}, create a copy using .copy(). Use del to remove "ID1" from the original dictionary and demonstrate that the copy is unaffected.	**Moderate**
19	Given a list of tuples representing student records students = [("Rahul", 85), ("Sriya", 40), ("Karan", 62)], write a function filter_passing(student_list) that loops through and returns a list of students who scored 50 or above.	**Moderate**
20	Write a program using a while loop that continuously takes numerical inputs from a user and appends them to a list. Stop when the user enters -1. Then, write a custom function find_highest(lst) that finds and returns the largest number in the list without using the built-in max().	**Moderate**

21. The Custom E-Commerce Price Tag Sorter

An online storefront stores product listings as a list of dictionaries:

products = [{"name": "Laptop", "price": 55000}, {"name": "Mouse", "price": 800}, {"name": "Keyboard", "price": 2500}].

- Write a program that asks the user whether they want to sort by "price" or by "name".
- Write two helper functions, sort_by_price(item) and sort_by_name(item).
- Pass the appropriate function to sorted() using the key parameter based on the user's choice.
- Print the newly sorted product list clearly.

22. The HR Payroll Salary Tax Adjuster

An HR system maintains employee salary tuples inside a dictionary:

payroll = {"E101": (50000, "Tech"), "E102": (75000, "Executive"), "E103": (30000, "Tech")}.

- Write a custom function calculate_tax(salary) that applies a 10% tax rate if salary is 50,000 or less, and 20% if above 50,000.
- Loop through the keys of payroll.
- Unpack each tuple into salary and dept.
- Calculate the net salary using calculate_tax() and update payroll[emp_id] to store a new tuple: (original_salary, net_salary, dept).

23. The Flight Reservation Seat Allocator

An airline tracks seat allocations using a dictionary with tuple keys representing (row_number, seat_letter):

flight_seats = {(1, "A"): "Occupied", (1, "B"): "Vacant", (2, "A"): "Occupied"}.

- Write a program to take row number and seat letter from a passenger.
- Form a tuple requested_seat = (row, seat).
- Check if requested_seat exists in flight_seats using the in operator.
- If it exists and status is "Vacant", update it to "Occupied" and print "Booking confirmed!".
- If it is already "Occupied", loop through flight_seats and print a list of all tuple keys where the status is "Vacant".

24. The Real-Time Analytics Pipeline

A telemetry system receives raw sensor data with corrupted negative readings:

raw_readings = [12.4, -999.0, 15.6, 18.2, -999.0, 22.1, 14.0].

- Write a function clean_and_round_data(data) that:
    1. Filters out all negative readings (< 0).
    2. Rounds all remaining valid readings to the nearest whole integer using round().
- Return and print the final cleaned list of readings.

25. The Multi-Branch Warehouse Inventory Merge

Two distribution centers track stock levels:

warehouse_A = {"Items_A": 150, "Items_B": 80, "Items_C": 40}

warehouse_B = {"Items_B": 50, "Items_C": 60, "Items_D": 200}.

- Create a duplicate copy of warehouse_A named master_stock.
- Loop through the keys of warehouse_B. If an item exists in master_stock, add the stock quantity from warehouse_B to the existing amount.
- If it doesn't exist, add the new item key-value pair.
- Write a custom key function get_stock(item_key) to sort and print master_stock items from highest stock to lowest stock using sorted().

26. The Fleet Delivery Route Optimizer

A courier service stores daily delivery coordinates as a list of tuples representing distance from hub in kilometers:

deliveries = [("Location A", 12.5), ("Location B", 3.2), ("Location C", 8.7), ("Location D", 1.5)].

- Write a helper function extract_distance(delivery_tuple) that returns the second element of the tuple.
- Sort the deliveries list in ascending order based on distance using .sort(key=extract_distance).
- Print the optimized trip schedule using a for loop.

27. The Student Performance Bracket Filter

A school administration records student performance as a list of dictionaries:

gradebook = [{"name": "Rohan", "score": 82}, {"name": "Pooja", "score": 45}, {"name": "Neha", "score": 91}, {"name": "Amit", "score": 38}].

- Write a function process_gradebook(records) that:
  - Builds a list retest_list containing student dictionaries for those who scored below 50.
  - Builds a list of formatted strings like "Rohan: PASS" (if score ≥ 50) or "Pooja: FAIL" (if score < 50).
- Return both lists from the function and print them.

28. The Smart Home Appliance Scheduler

A smart hub controls household devices:

devices = {"Light": "OFF", "AC": "ON", "Fan": "ON", "Heater": "OFF"}.

- Write a function toggle_device_status(current_status) that turns "ON" to "OFF" and "OFF" to "ON".

- Loop over the device names (keys) in devices.
- If the user chooses to "Toggle All", call toggle_device_status() to update every device state inside the dictionary.
- Print the final updated status of all smart home devices.

29. The Security Log Threat Detector

A cybersecurity server records incoming request metadata as tuples:

logs = [("192.168.1.1", "GET", 200), ("10.0.0.5", "POST", 500), ("192.168.1.1", "POST", 403), ("172.16.0.2", "GET", 500)].

- Write a function detect_server_errors(log_list) that loops through the logs and extracts all tuples where the HTTP status code (3rd element) is 500.
- Count how many internal server errors (code 500) occurred.
- Print the isolated error logs along with the total count.

30. The Digital Wallet Transaction Ledger

A digital wallet tracks transaction history as a list of tuples containing transaction type and amount:

transactions = [("Deposit", 1000), ("Withdrawal", 200), ("Withdrawal", 150), ("Deposit", 500)].

- Write a function process_ledger(transaction_list) that starts with balance = 0.
- Loop through transactions, unpacking each tuple into t_type and amount.
- Add deposits and subtract withdrawals from balance.
- Write a second function is_solvent(balance) that returns True if balance >= 0 and False otherwise.
- Print the final balance and the result of is_solvent(balance).